

Android 中声明式 UI、组合与继承、重组的核心思想

Android 教学讲义：声明式UI、组合vs继承、重组

一、什么是声明式UI？

小白理解：就像写作文描述"最终样子"，而不是写"步骤"。

对比传统方式（命令式）和声明式：

命令式（比如传统Android的XML+Java）：

Code block

```
1  // 步骤1: 创建按钮
2  Button button = new Button(context);
3  // 步骤2: 设置文字
4  button.setText("点击我");
5  // 步骤3: 设置颜色
6  button.setTextColor(Color.RED);
7  // 步骤4: 添加到界面
8  container.addView(button);
```

思考：如果要改文字，还得再写一行 `button.setText("新文字")`，每次变化都要手动操作

声明式（比如Jetpack Compose）：

Code block

```
1  @Composable
2  fun MyButton() {
3      Button(
4          text = "点击我",
5          textColor = Color.Red
6      )
7  }
```

思考：直接描述按钮"应该是什么样"，如果数据变了，界面自动更新，不用手动改

核心区别：

命令式：告诉系统"怎么做"（步骤）

声明式：告诉系统"是什么"（结果）

二、组合 vs 继承：哪种更好？

继承（Inheritance）：像"儿子继承父亲的财产"

Code block

```
1  // 父类：基础按钮
2  class BaseButton : Button {
3      fun setBaseStyle() {
4          textSize = 16.sp
5          backgroundColor = Color.Blue
6      }
7  }
8
9  // 子类：继承后加新功能
10 class SpecialButton : BaseButton {
11     fun addIcon() {
12         // 在父类基础上添加图标
13     }
14 }
```

问题：如果要同时有"蓝色按钮"和"带图标的按钮"，需要创建多个子类，越复杂越乱（称为"类爆炸"）

组合（Composition）：像"搭积木"

Code block

```
1  // 积木1：蓝色样式
2  @Composable
3  fun BlueStyle(content: @Composable () -> Unit) {
4      Box(modifier = Modifier.background(Color.Blue)) {
5          content() // 包裹传入的内容
6      }
7  }
8
9  // 积木2：带图标
10 @Composable
11 fun WithIcon(icon: Int, content: @Composable () -> Unit) {
12     Row {
13         Icon(imageVector = Icons.Default.Info)
```

```

14         content() // 包裹传入的内容
15     }
16 }
17
18 // 组合使用：想要"蓝色+带图标+文字"的按钮
19 @Composable
20 fun MySpecialButton() {
21     BlueStyle {
22         WithIcon(icon = Icons.Default.Info) {
23             Text("我是组合按钮")
24         }
25     }
26 }

```

优势：想用什么功能就组合什么，不用创建一堆子类，灵活得多！

总结：

继承：父子关系，不灵活，容易复杂

组合：积木关系，灵活，可复用性强

Android推荐用**组合**（尤其是Compose中，完全基于组合思想）

三、什么是重组（Recomposition）？

小白理解：声明式UI中，数据变了，界面自动"重新画"需要更新的部分（不是整个界面都重画）

举个例子：

Code block

```

1  @Composable
2  fun Counter() {
3      var count by remember { mutableStateOf(0) } // 可观察的数据
4
5      Column {
6          Text("当前计数: $count") // 用到count的地方
7          Button(onClick = { count++ }) {
8              Text("点我加1")
9          }
10         Text("这段文字不依赖count") // 不依赖count的地方
11     }
12 }

```

当点击按钮，`count` 变成1时：

- 重组会**只更新**用到 `count` 的 `Text("当前计数: $count")`

- 按钮和另一段文字**不会重新绘制**

重组的好处：

只更新需要变的部分，性能更高，比传统的 `notifyDataSetChanged()`（刷新整个列表）高效得多。

四、一句话总结

1. 声明式UI：描述"界面是什么样"，而非"怎么做"
2. 组合vs继承：组合像搭积木（推荐），继承像父子传代（不灵活）
3. 重组：数据变了，自动更新用到该数据的界面部分，性能好

通过这三个概念，你就能理解现代Android开发（如Jetpack Compose）的核心思想啦！

（注：文档部分内容可能由 AI 生成）